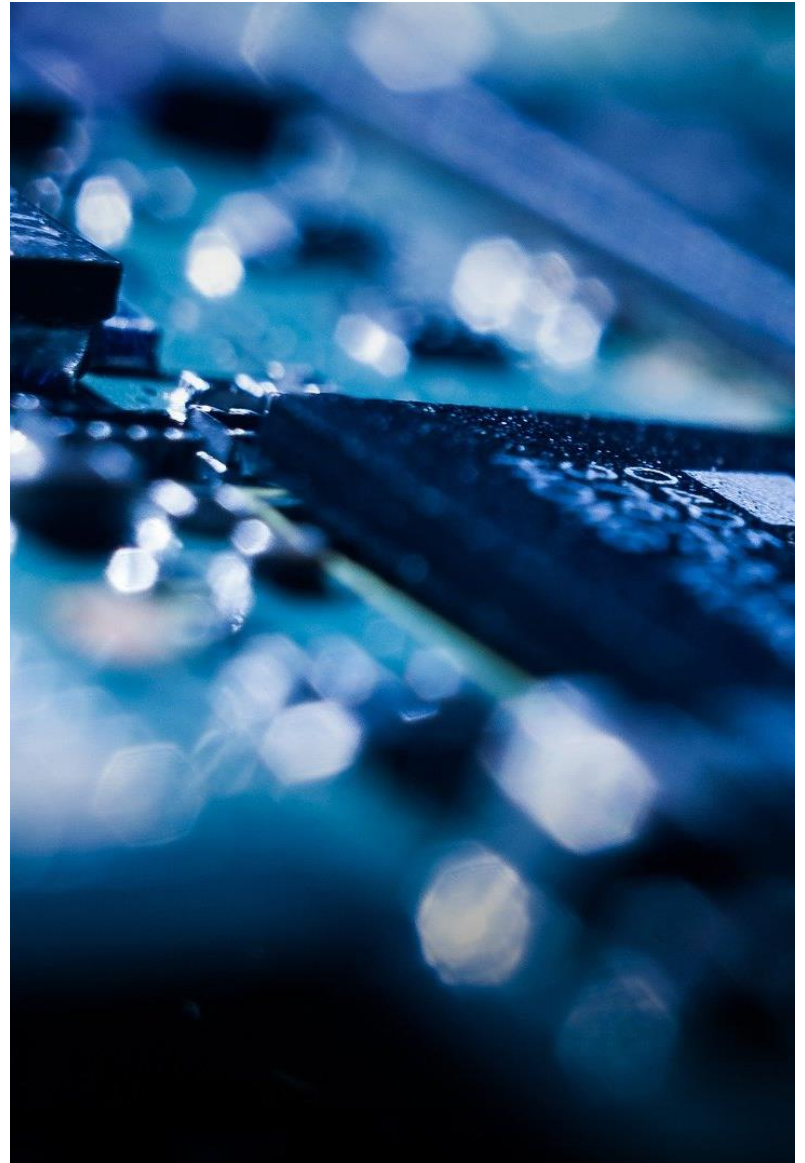


Programmierung eingebetteter Systeme

Felder und Strukturen

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



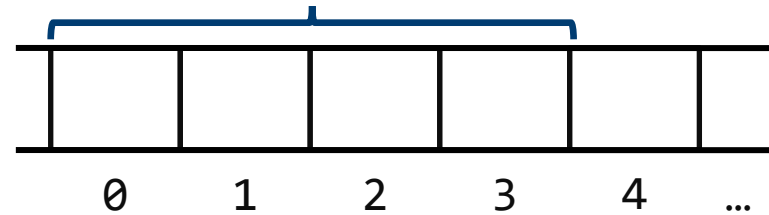
Überblick – Felder und Strukturen

- ▶ Felder allgemein
- ▶ Felder mit char-Variablen
- ▶ Strukturen allgemein
- ▶ Strukturen statisch und dynamisch im Speicher ablegen
- ▶ Strukturprototypen
- ▶ Anwendung von Struktur-Masken

Zusammengesetzte Variablen, Felder

Felder: Variablen gleichen Typs können über **einen** Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden

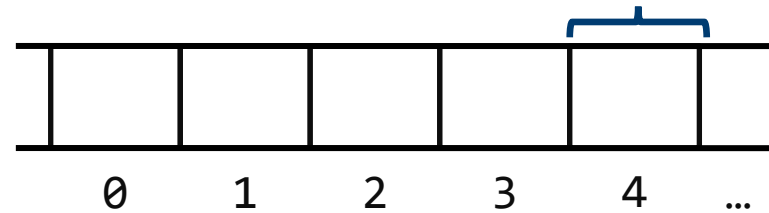
```
int array[4]; ←  
int count;  
...  
count = 5;  
...  
array[0] = 10;  
array[1] = 11;  
array[2] = 12;  
array[3] = 13;
```



Zusammengesetzte Variablen, Felder

Felder: Variablen gleichen Typs können über **einen** Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden

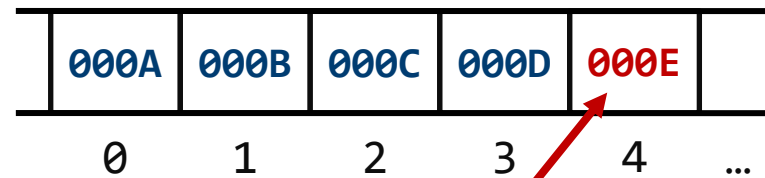
```
int array[4];  
int count; ←  
...  
count = 5;  
...  
array[0] = 10;  
array[1] = 11;  
array[2] = 12;  
array[3] = 13;
```



Zusammengesetzte Variablen, Felder

Felder: Variablen gleichen Typs können über **einen** Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden

```
int array[4];  
int count;  
...  
count = 5;  
...  
array[0] = 10;  
array[1] = 11;  
array[2] = 12;  
array[3] = 13;
```



array[4] = 14;

Achtung! Nächste Variable im Speicher wird überschrieben.

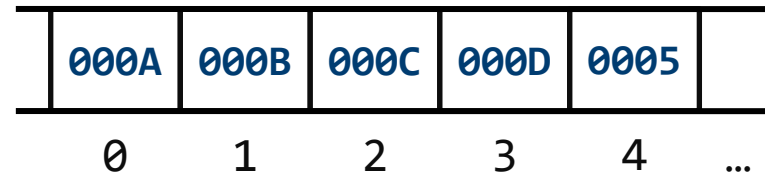
Feldelemente beginnen mit Index 0

Kein Schutz vor Bereichsüberschreitung!

Zusammengesetzte Variablen, Felder

Felder: Variablen gleichen Typs können über **einen** Namen und den Feldindex (Nummerierung der Feldelemente) erreicht werden

```
int array[4];  
int count;  
int loop;  
...  
count = 5;  
...  
for(loop = 0; loop < 4; loop++)  
{  
    array[loop] = loop + 10;  
}
```

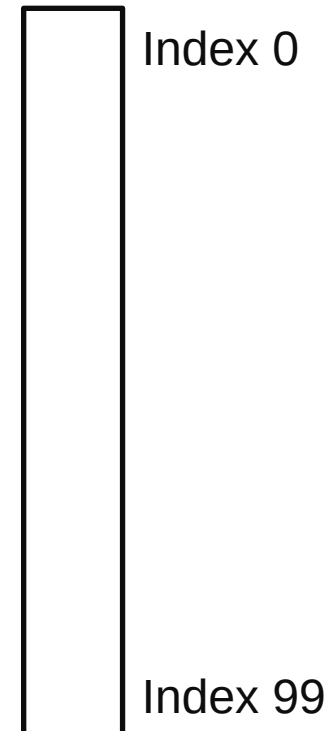


Zusammengesetzte Variablen, Felder

Beispiel: 100 Messwerte sammeln

```
int array[100];
int index;
...

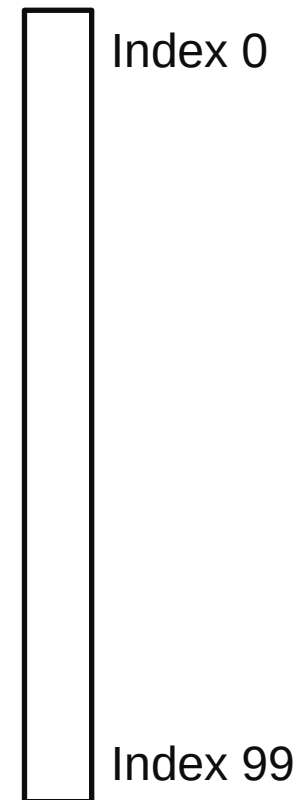
for(index = 0; index < 100; index++)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
}
...
```



Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

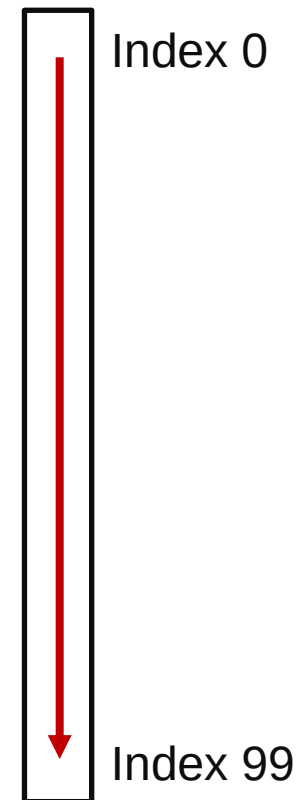
```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```



Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

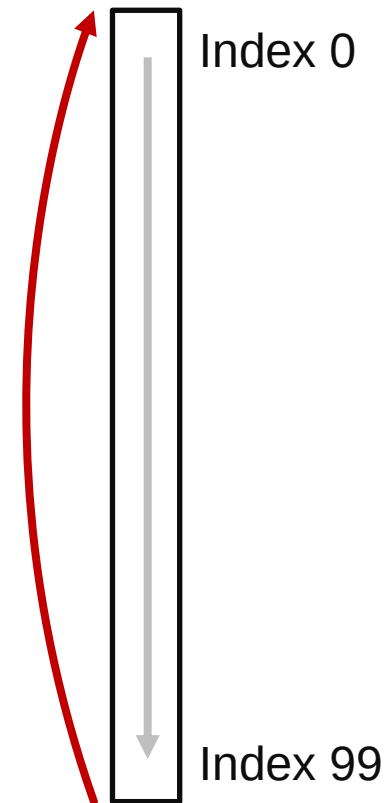
```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```



Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

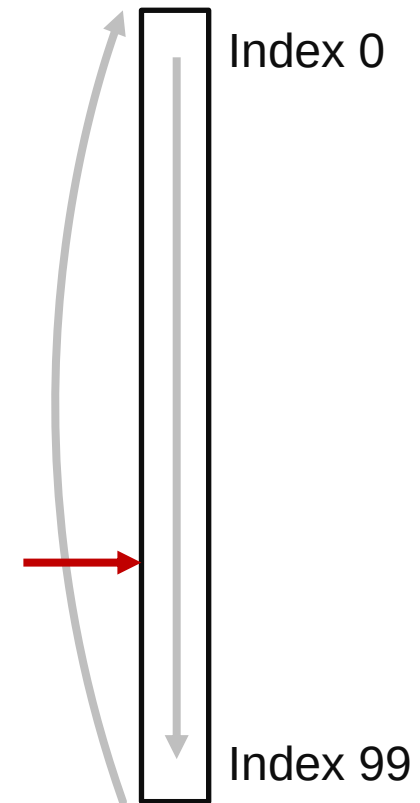
```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```



Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```

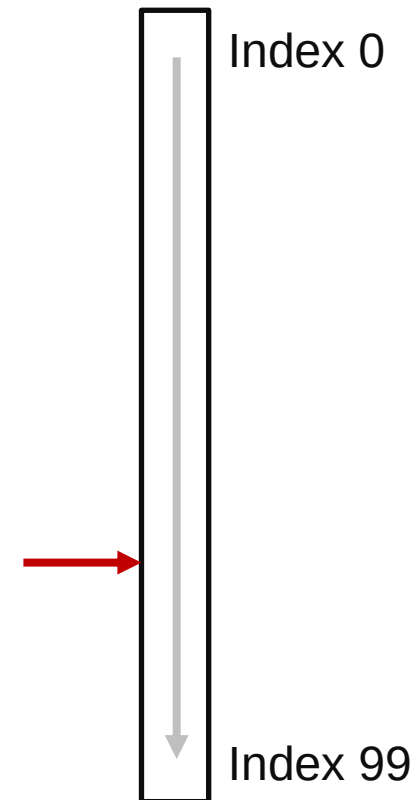


Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```

**Werte lesen
z.B. neuester
Wert zuerst**

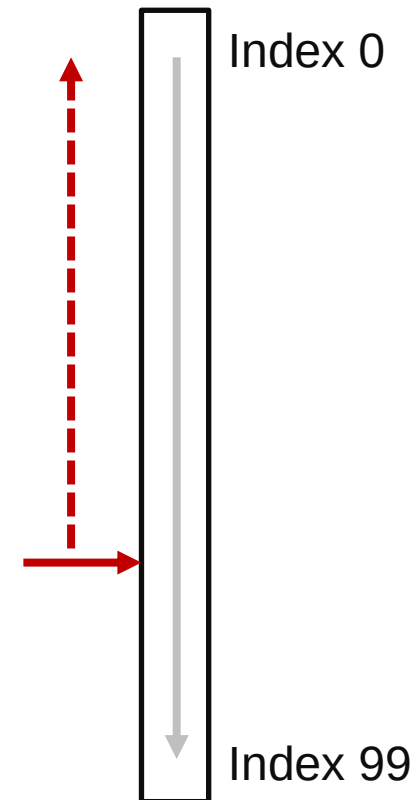


Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```

**Werte lesen
z.B. neuester
Wert zuerst**

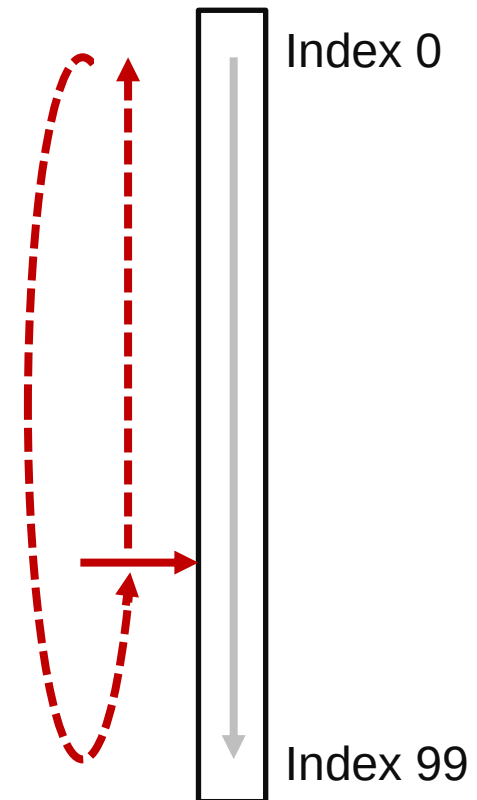


Zusammengesetzte Variablen, Felder

Beispiel: Ständig Messwerte sammeln, bis ein Ereignis eintritt. Die letzten 100 Messwerte speichern.

```
int array[100];
int index;
int erster_durchlauf;
...
erster_durchlauf = 1;
index = 0;
for(;;)
{
    ...
    neuer_messwert = ... ;
    array[index] = neuer_messwert;
    ...
    if(ereignis){ break; }
    if(index == 99){ index = 0; erster_durchlauf = 0; }
    else{ index++; }
}
```

Werte lesen
z.B. neuester
Wert zuerst



Felder mit char-Variablen

Zusammengesetzte Variablen, Felder

Felder: Character-Felder („String-Variablen“)

```
char xy[30];  
...  
xy[0] = 't';  
xy[1] = 'e';  
xy[2] = 's';  
xy[3] = 't';  
xy[4] = 0x00;
```

't'	'e'	's'	't'	00	
0	1	2	3	4	...

```
int loop;  
...  
for(loop = 0; loop < 10; loop++)  
{  
    xy[loop] = (char)(loop + 1);  
}  
...
```

01	02	03	04	05	
0	1	2	3	4	...

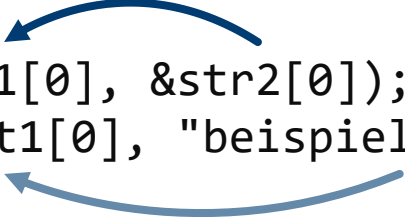
Zusammengesetzte Variablen, Felder

Character-Felder („String-Variablen“), ANSI-Library "string.h"

Funktionalität:

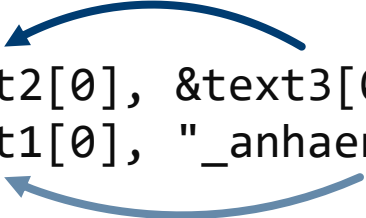
Kopieren von Strings

Beispiele:



```
strcpy(&str1[0], &str2[0]);  
strcpy(&text1[0], "beispiel_text");
```

Anhängen eines Strings
an einen anderen



```
strcat(&text2[0], &text3[0]);  
strcat(&text1[0], "_anhaengen");
```

text1: beispiel_text_anhaengen

Zusammengesetzte Variablen, Felder

Beispiel 1: Zeiger auf Felder

```
short int array_bsp[50];
```

 **short int: 2 Byte * 50 Elemente**

```
short int *array_ptr;
```

```
...
```

```
array_ptr = &array_bsp[0];
```

```
for(i=0; i<50; i++)
```

```
{
```

```
    *array_ptr = ...;
```

```
    array_ptr++;
```

```
}
```

Zusammengesetzte Variablen, Felder

Beispiel 1: Zeiger auf Felder

```
short int array_bsp[50];  
short int *array_ptr; ← Pointer auf short int: 2 Byte  
...  
array_ptr = &array_bsp[0];  
  
for(i=0; i<50; i++)  
{  
    *array_ptr = ...;  
    array_ptr++;  
}
```

Zusammengesetzte Variablen, Felder

Beispiel 1: Zeiger auf Felder

```
short int array_bsp[50];
```

```
short int *array_ptr;
```

```
...
```

```
array_ptr = &array_bsp[0];
```

← **Pointer auf Feldanfang (Index 0)**

```
for(i=0; i<50; i++)
```

```
{
```

```
    *array_ptr = ...;
```

```
    array_ptr++;
```

```
}
```

Zusammengesetzte Variablen, Felder

Beispiel 1: Zeiger auf Felder

```
short int array_bsp[50];
```

```
short int *array_ptr;
```

```
...
```

```
array_ptr = &array_bsp[0];
```

```
for(i=0; i<50; i++)
```

```
{
```

```
    *array_ptr = ...; ← Feldelement mit Wert füllen
```

```
    array_ptr++;
```

```
}
```

Zusammengesetzte Variablen, Felder

Beispiel 1: Zeiger auf Felder

```
short int array_bsp[50];
```

```
short int *array_ptr;
```

```
...
```

```
array_ptr = &array_bsp[0];
```

```
for(i=0; i<50; i++)
```

```
{
```

```
    *array_ptr = ...;
```

```
    array_ptr++;
```

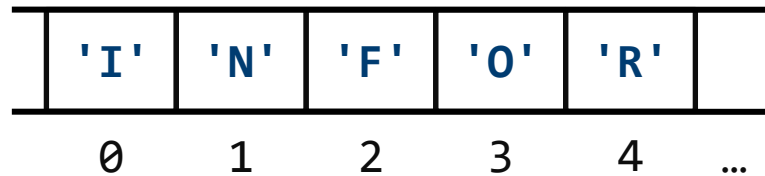
```
}
```

**← Pointer um eine Objektlänge
(hier: 2 Byte)iterrücken**

Zusammengesetzte Variablen, Felder

Beispiel 2: Zeiger auf Felder

```
char array_bsp[50];  
char *array_ptr;  
...  
array_ptr = &array_bsp[0];
```

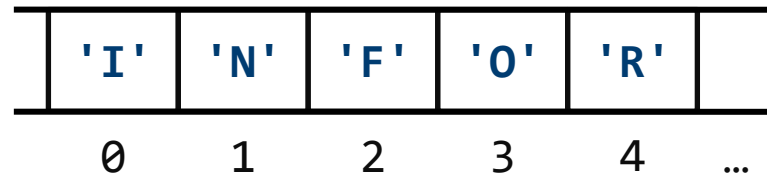


```
for(i=0; i<50; i++)  
{  
    if(*array_ptr == 'F')  
    {  
        break;  
    }  
    array_ptr++;  
}
```

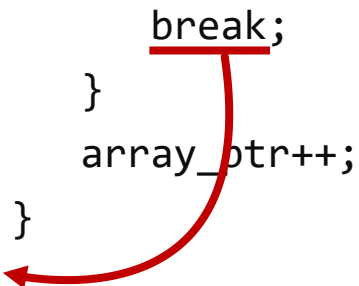
Zusammengesetzte Variablen, Felder

Beispiel 2: Zeiger auf Felder

```
char array_bsp[50];  
char *array_ptr;  
...  
array_ptr = &array_bsp[0];
```



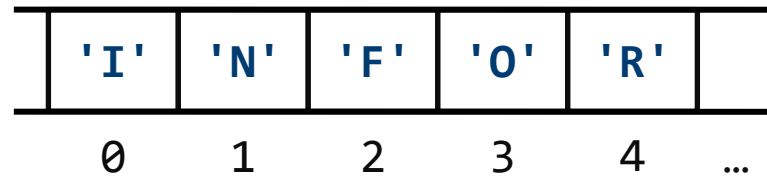
```
for(i=0; i<50; i++)  
{  
    if(*array_ptr == 'F')  
    {  
        break;  
    }  
    array_ptr++;  
}
```



Zusammengesetzte Variablen, Felder

Beispiel 2: Zeiger auf Felder

```
char array_bsp[50];  
char *array_ptr;  
...  
array_ptr = &array_bsp[0];
```



```
for(i=0; i<50; i++)  
{  
    if(*array_ptr == 'F')  
    {  
        break;  
    }  
    array_ptr++;  
}
```

**array_ptr zeigt nach Abarbeitung
der Schleife auf 'F'**

Strukturen allgemein

Strukturen

Struktur in C

- zusammengesetzte Variable
- feste Anzahl von Komponenten, die unterschiedliche Datentypen besitzen können

Strukturen

Beispiel: Struktur „Student“

Name	char(30)
Vorname	char(15)
Matrikelnummer	long int
Postleitzahl	long int
Ort	char(20)
Strasse	char(25)
Hausnummer	int

```
typedef struct stud
{
    char name[30];
    char vname[15];
    long int matr;
    long int plz;
    char ort[20];
    char strasse[25];
    int hausnr;
} Student;

Student xyz;
```

Strukturen

Beispiel: Struktur „Student“

Name	char(30)	typedef struct stud
Vorname	char(15)	{
Matrikelnummer	long int	char name[30];
Postl	bekannter Datentyp unsigned char	char vname[15];
Ort	char(20)	long ort[20];
Strasse	typedef unsigned char Byte;	long strasse[25];
Hausnummer	Byte var;	ausnr;
		t;
		Student xyz;

neuer Datentyp Byte

Variable var vom Typ Byte

Strukturen

Beispiel: Struktur „Student“

Name	char(30)
Vorname	char(15)
Matrikelnummer	long int
Postleitzahl	long int
Ort	char(20)
Strasse	char(25)
Hausnummer	int

neuer
Datentyp
Student

'bekannter' Datentyp
struct stud

```
typedef struct stud  
{  
    char name[30];  
    char vname[15];  
    long int matr;  
    long int plz;  
    char ort[20];  
    char strasse[25];  
    int hausnr;  
};
```

Student;

Student xyz;

Variable xyz
vom Typ
Student

Strukturen

Beispiel: Struktur „Student“

sinnvoll ist hier: Feld von
Strukturen (Meier, Müller, ...)

Name	char(30)
Vorname	char(15)
Matrikelnummer	long int
Postleitzahl	long int
Ort	char(20)
Strasse	char(25)
Hausnummer	int

wie bisher

```
typedef struct stud
{
    char name[30];
    char vname[15];
    long int matr;
    long int plz;
    char ort[20];
    char strasse[25];
    int hausnr;
} Student;
```

```
Student xyz[100];
```

Feld xyz: jedes Feldelement vom Typ Student

Strukturen

Beispiel: Struktur „Student“

Name

char(30)

typedef struct stud

Feldindex 0	char name[30];	char vname[15];	long int matr;	long int plz;	...
--------------------	----------------	-----------------	----------------	---------------	-----

Feldindex 1	char name[30];	char vname[15];	long int matr;	long int plz;	...
--------------------	----------------	-----------------	----------------	---------------	-----

Feldindex 2	char name[30];	char vname[15];	long int matr;	long int plz;	...
--------------------	----------------	-----------------	----------------	---------------	-----

Feldindex 3	char name[30];	char vname[15];	long int matr;	long int plz;	...
--------------------	----------------	-----------------	----------------	---------------	-----

...	...	...	...	...	...
-----	-----	-----	-----	-----	-----

Feldindex 99	char name[30];	char vname[15];	long int matr;	long int plz;	...
---------------------	----------------	-----------------	----------------	---------------	-----

Student xyz[100];

Strukturen statisch und dynamisch im Speicher anlegen

Struktur statisch im Speicher anlegen

```
...
int loop;
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Neu;

Neu y[10];
...
```

```
...
void main(void)
{
    ...
    for(;;)
    {
        ...
        for(loop = 0; loop < 10; loop++)
        {
            y[loop].a = ...;
            y[loop].b = ...;
            strcpy(&(y[loop].name[0]), "beispiel");
            ...
        }
        ...
    }
}
```

Struktur statisch im Speicher anlegen

```
...  
int loop;  
typedef struct x  
{  
    int a;  
    int b;  
    char name[25];  
    ...  
} Neu;
```

```
Neu y[10];
```

**Feld aus Strukturen
(Feld y vom Typ Neu)**

```
...  
void main(void)  
{  
    ...  
    for(;;)  
    {  
        ...  
        for(loop = 0; loop < 10; loop++)  
        {  
            y[loop].a = ...;  
            y[loop].b = ...;  
            strcpy(&(y[loop].name[0]), "beispiel");  
            ...  
        }  
        ...  
    }  
}
```

Struktur statisch im Speicher anlegen

Speicherplatz wird vom Compiler / Linker festgelegt

```
...  
int loop;  
typedef struct x  
{  
    int a;  
    int b;  
    char name[25];  
    ...  
} Neu;  
  
Neu y[10];  
...
```

```
...  
void main(void)  
{  
    ...  
    for(;;)  
    {  
        ...  
        for(loop = 0; loop < 10; loop++)  
        {  
            y[loop].a = ...;  
            y[loop].b = ...;  
            strcpy(&(y[loop].name[0]), "beispiel");  
            ...  
        }  
        ...  
    }  
}
```

Struktur dynamisch im freien Speicher anlegen

```
// definitions
#define ARRAY 10
#define MEM_START 0x20000000

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Messwert;

// module variables
int loop;
Messwert *mess;
```

```
void main(void)
{
    for(;;)
    {
        mess = (Messwert*)MEM_START;

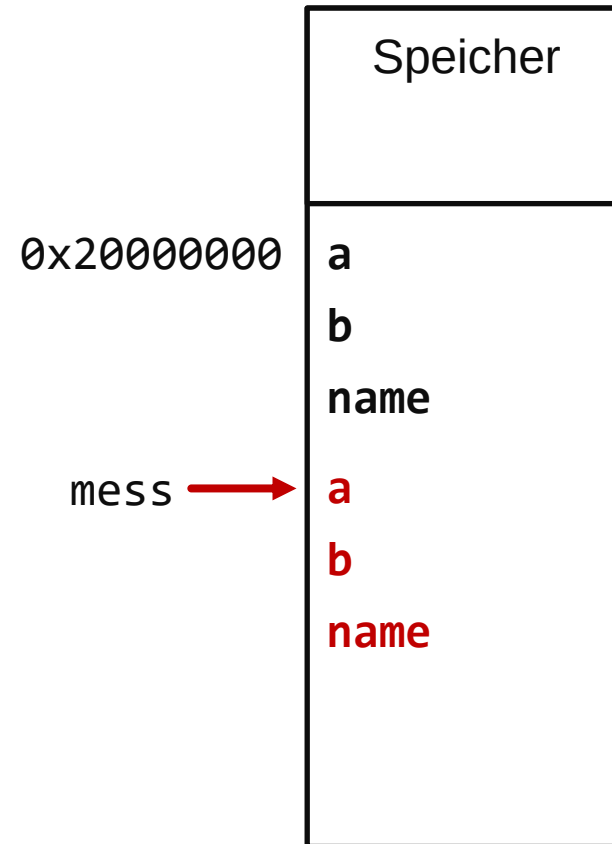
        for(loop = 0; loop < ARRAY; loop++)
        {
            mess->a = ...;
            mess->b = ...;
            ...
            mess++;
        }
        ...
    }
}
```

Struktur dynamisch im freien Speicher anlegen

```
// definitions
#define ARRAY 10
#define MEM_START 0x20000000
```

```
// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Messwert;
```

```
// module variables
int loop;
Messwert *mess;
```



Beispielaufgabe

Ein Datensatz besteht aus vier Komponenten mit den Datentypen `char`, `int`, `float` und `Pointer auf float`. Sie wollen fünf dieser Datensätze hintereinander im Speicher ablegen und wissen, dass ab `0x2A000000` Platz im Speicher ist. Die ersten drei Komponenten sollen mit 0 initialisiert werden, der `Pointer` soll auf die `float`-Komponente des gleichen Datensatzes zeigen.

Wie sieht das Programm aus?

Lösung

```
typedef struct beispiel
{
    char komponente1;
    int komponente2;
    float komponente3;
    float * komponente4;
} Beispiel;

Beispiel * ptr;
int loop;

...

ptr = (Beispiel *) 0x2A000000;

for (loop = 0, loop < 5; loop++)
{
    ptr->komponente1 = 0;
    ptr->komponente2 = 0;
    ptr->komponente3 = 0.0;
    ptr->komponente4 = &(ptr->komponente3);
    ptr++;
}
```


Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10
```

```
// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;
```

```
// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;
```

neuer Datentyp
als Schablone
(Struktur-Maske)

kein Speicher
reserviert

Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;
```

```
// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;
```

Anfangsadresse für Schablone

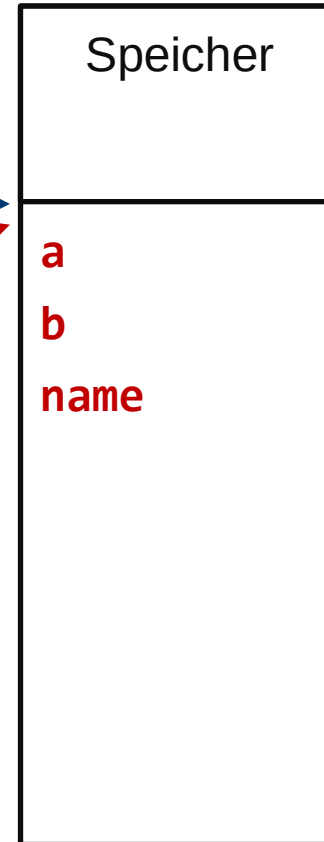


Struktur dynamisch im verw. Speicher anlegen

```
// definitions  
#define ARRAY 10
```

```
// prototypes  
typedef struct x  
{  
    int a;  
    int b;  
    char name[25];  
    ...  
} Mess;
```

```
// module variables  
int loop;  
void *mem_ptr;  
Mess *mess_ptr;
```



Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;

void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*(sizeof(Mess))));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < ARRAY; loop++)
        {
            mess_ptr->a = ...;
            mess_ptr->b = ...;
            strcpy(&(mess_ptr->name[0]), "beispiel");
            ...
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```

**memory allocation
(Speicher holen)**

Struktur dynamisch im verw. Speicher anlegen

Reservierung von Speicher

Der Compiler/Linker erzeugt Speicherbereiche für Programmcode, Daten und Stack. Zusätzlich wird Speicherplatz zur Schaffung dynamischer Variablen bereitgestellt (Heap). Als Zusatzinfo für den Linker muss der Programmierer die Speicherbereiche für nichtveränderliche Objekte (Programmcode) und für veränderliche Objekte (Variablen, Stack, Heap) bekanntgeben.

Reservierung von Speicher im Heap erfolgt mit der Library-Funktion **memory allocation malloc()**. Ein solches Speicherobjekt existiert bis zur Rückgabe mit **free()**.

Prototyp: **void* malloc(size_t size);**

Die Funktion **malloc()** gibt einen Zeiger auf den reservierten Speicherbereich zurück, wenn die Speicherreservierung durchgeführt werden konnte, sonst wird **NULL** zurückgegeben. Der Rückgabewert ist vom Typ 'Zeiger auf **void**'. Als Parameter benötigt die Funktion die Speichergröße (Anzahl Bytes).

Struktur dynamisch im verw. Speicher anlegen

Reservierung von Speicher

Beispiel: Speicher von der Länge 100 Bytes holen und für

Reservierung allocation mit free().	<pre> ... void *memory_ptr; ... memory_ptr = malloc(100); </pre>	memory ckgabe mit
---	--	----------------------

Prototyp: vo ...

Die Funktion `malloc()` gibt einen Zeiger auf den reservierten Speicherbereich zurück, wenn die Speicherreservierung durchgeführt werden konnte, sonst wird `NULL` zurückgegeben. Der Rückgabewert ist vom Typ 'Zeiger auf `void`'. Als Parameter benötigt die Funktion die Speichergröße (Anzahl Bytes).

Struktur dynamisch im verw. Speicher anlegen

Das Bestimmen der erforderlichen Speichergröße durch den Programmierer ist sehr fehleranfällig. Es ist sinnvoll, die Zahl der Bytes mit Hilfe des **sizeof**-Operators zu bestimmen.

Der Operator **sizeof** wird auf einen Datentyp angewandt und liefert die Größe des angegebenen Datentyps gemessen in Bytes.
Der Rückgabewert ist vom Typ **size_t**.

Beispiel: `anzahl = sizeof(float);`
Der Variablen `anzahl` wird die Zahl 4 zugewiesen.

Die Größe einer Struktur lässt sich i.A. nicht durch die Addition der Größen der Komponenten ermitteln, da der Compiler die Strukturkomponenten auf bestimmten Adressen beginnen lässt, z.B. **WORD** und **LONG**-Objekte nur auf Wortgrenzen (Adressen: `xxx0`, `xxx2`, `xxx4` usw). Eine solche Anordnung von Objekten im Speicher wird als **Alignment** bezeichnet.

```
typedef struct { ... }Mess;  
mem_ptr = malloc((size_t)(LENGTH_OF_ARRAY * (sizeof(Mess))));
```

Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;
```

```
void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*sizeof(Mess)));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < ARRAY; loop++)
        {
            mess_ptr->a = ...;
            mess_ptr->b = ...;
            strcpy(&(mess_ptr->name[0]), "beispiel");
            ...
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```

Pointer auf den Anfang des reservierten Speicherbereichs (void *)

Länge einer Struktur

Länge des Feldes aus Strukturen

Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;
```

```
void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*sizeof(Mess)));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < ARRAY; loop++)
        {
            ...
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```

Pointer auf den Anfang des reservierten Speicherbereichs (void *)

Länge einer Struktur

Länge des Feldes aus Strukturen

Pointer auf "Messwert" im freien Speicher
Compiler "kennt" die Offsets der Strukturelemente

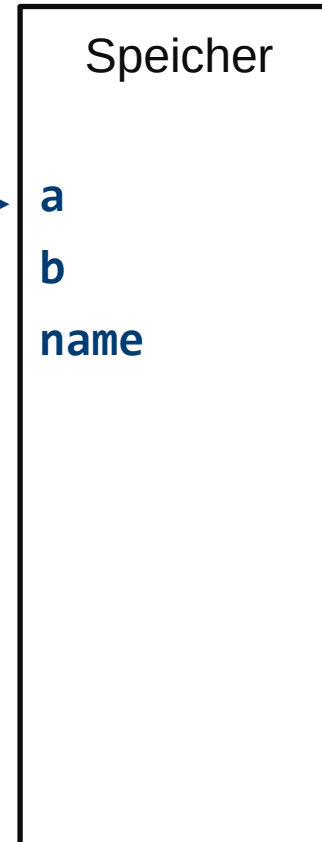
Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;
```

Pointer
mess



Adresse wird
von malloc
geliefert

Struktur **dynamisch** im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;

void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*(sizeof(Mess))));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < AR
        {
            mess_ptr->a = ...;
            mess_ptr->b = ...;
            strcpy(&(mess_ptr->name[0]), "beispiel");
            ...
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```

**Zugriff auf
Strukturelement
über Pointer**

Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;

void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*(sizeof(Mess))));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < ARRAY; loop++)
        {
            mess_ptr->a = ...;
            mess_ptr->b = ...;
            strcpy(&(mess_ptr->name[0]), "beispiel");
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```

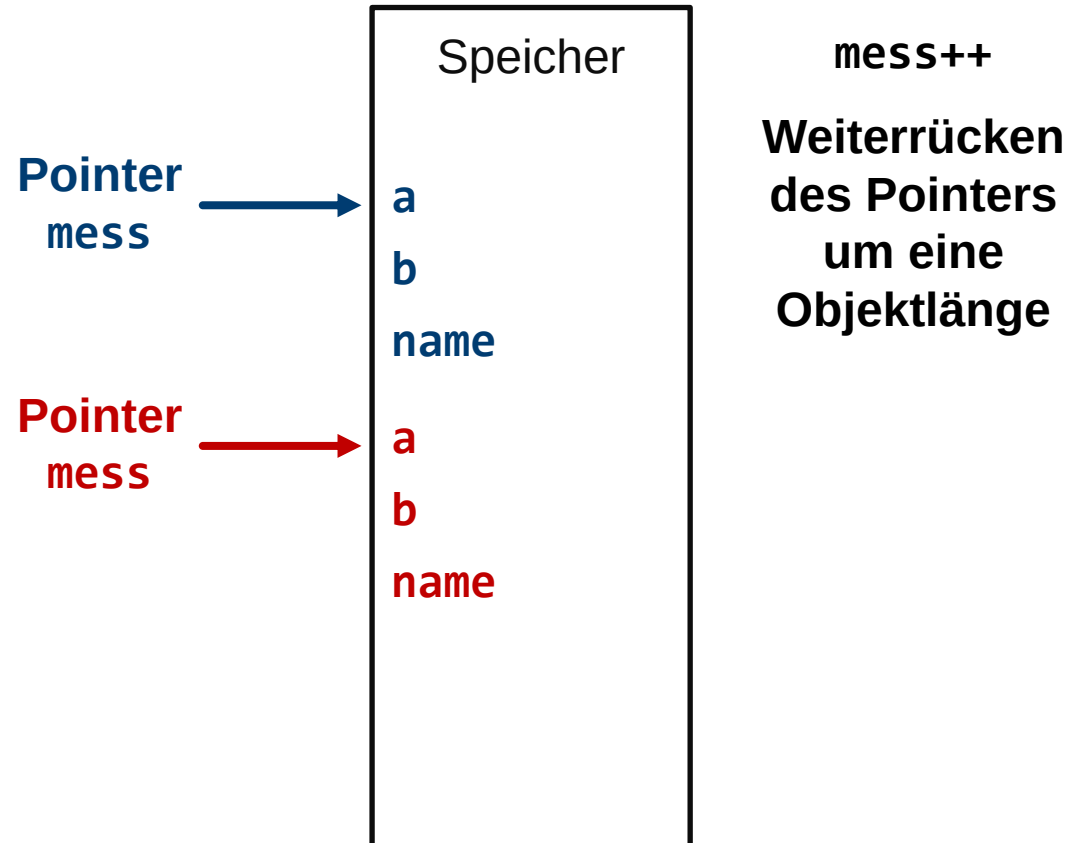
Pointer weiter rücken

Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;
```



Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;

void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*(sizeof(Mess))));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < ARRAY; loop++)
        {
            mess_ptr->a = ...;
            mess_ptr->b = ...;
            strcpy(&(mess_ptr->name[0]),"beispiel");
            ...
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```



Struktur dynamisch im verw. Speicher anlegen

```
// definitions
#define ARRAY 10

// prototypes
typedef struct x
{
    int a;
    int b;
    char name[25];
    ...
} Mess;

// module variables
int loop;
void *mem_ptr;
Mess *mess_ptr;

void main(void)
{
    ...
    for(;;)
    {
        mem_ptr = malloc((size_t)(ARRAY*(sizeof(Mess))));
        mess_ptr = (Mess*)mem_ptr;
        ...
        for(loop = 0; loop < ARRAY; loop++)
        {
            mess_ptr->a = ...;
            mess_ptr->b = ...;
            strcpy(&(mess_ptr->name[0]),"beispiel");
            ...
            mess_ptr++;
        }
        ...
        free(mem_ptr);
    }
}
```